

Lab 8: Strings (with Pointers)

Computer Programming () |

Duration: 2 hours (in-lab) | **Topic:** Null-terminated `char` arrays, `char *` traversal, `string.h`

Objectives

- Represent strings as null-terminated `char` arrays.
- Use both indexing (`s[i]`) and pointer traversal (`*p, p++`) on strings.
- Read strings safely with `scanf("%s", ...)` and `fgets`.
- Use `strlen`, `strcpy`, `strcmp`, `strcat`, and understand why `a == b` does not compare strings.

Quick Reference

A C string is a `char` array ending in `'\0'`.

```
char s[20] = "hello"; /* 6 bytes: 'h','e','l','l','o','\0' */
```

Two equivalent traversals.

```
/* index style */          /* pointer style */
for (int i = 0; s[i]; i++) ... for (char *p = s; *p; p++) ...
```

Both stop at the null terminator. They produce identical results because $s[i] \equiv *(s + i)$.

Reading input. `scanf("%s", s);` reads one whitespace-delimited word and writes `'\0'`.

`fgets(s, sizeof s, stdin);` reads a whole line including `'\n'`.

Library functions (`<string.h>`):

<code>strlen(s)</code>	number of chars before the <code>'\0'</code>
<code>strcpy(dst, src)</code>	copy (dst must be large enough)
<code>strcmp(a, b)</code>	0 if equal, < 0 if <code>a < b</code> , > 0 otherwise
<code>strcat(dst, src)</code>	append <code>src</code> onto <code>dst</code>

Pitfall. `a == b` compares *pointers*, not contents. Use `strcmp`.

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: my_strlen – index and pointer versions (25 min)

Write two functions that compute the length of a string *without* calling `strlen`:

- `int len_idx(const char s[])` – uses `s[i]` and an index.
- `int len_ptr(const char *s)` – walks a pointer `p` until `*p == '\0'`.

Verify both agree with `strlen` for at least three test inputs.

In-Lab Exercise 2: Reverse a String In Place (two pointers) (30 min)

Write `void reverse(char *s)` that reverses a string in place using two pointers (one starting at the beginning, one at the character before the `'\0'`). Test with input read via `scanf("%s", ...)`.

In-Lab Exercise 3: Count Vowels and Consonants (25 min)

Read a line with `fgets` and count vowels (case-insensitive), consonants, and other characters. Use pointer traversal: `for (char *p = line; *p; p++) ....`

In-Lab Exercise 4: Palindrome with Two Pointers (40 min)

Write `int is_palindrome(const char *s)` that returns 1 if `s` reads the same forward and backward (case-insensitive, ignoring spaces). Use two pointers `left` and `right` moving toward the middle. Test on "radar", "A man a plan a canal Panama", and "hello".

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Null terminator inside the buffer

What does this print?

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char s[10] = {'C', 'S', '\0', '1', '0', '1', '\0'};
    printf("%lu\n", (unsigned long)strlen(s));
    printf("%s\n", s);
    return 0;
}
```

Trace & Predict 2: Pointer-walk equivalence

Predict the output of both loops:

```
#include <stdio.h>
int main(void) {
    char s[] = "abc";
    for (int i = 0; s[i]; i++)        printf("%c", s[i]);
    printf(" / ");
    for (char *p = s; *p; p++)        printf("%c", *p);
    printf("\n");
    return 0;
}
```

Find the Bug 1: Comparing strings with == and buffer too small

List every bug and rewrite the corrected program.

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char a[3];
    char b[] = "yes";
    scanf("%s", a);
    if (a == b) printf("match\n");
    else       printf("no match\n");
    return 0;
}
```

Write Code on Paper 1: Uppercase via pointer

Write `void to_upper(char *s)` that walks the string with a pointer and converts every lowercase letter to uppercase. Do not use `ctype.h`. Then write a `main` that reads a word and prints it uppercased.

Write Code on Paper 2: Count occurrences of a character

Write `int count_char(const char *s, char c)` that returns the number of times `c` occurs in `s` (use pointer traversal). Demonstrate from `main` on input read with `fgets`.